



ENGINEERING GUIDE

Engineering Toolkit

Repeatable camera, routing, tracing, analysis, and native-research workflows.

2026 EDITION / IAIN BENNETT / MIT LICENSE

The VirtualGlove Engineering Toolkit is an optional, version-matched collection for people who want to measure, reproduce, or extend the camera-to-game pipeline. It is not required to install VirtualGlove, personalise a hand, adjust movement reach, or play a game.

The toolkit favours repeatable evidence over live guesswork. Most investigations begin with an existing trace or recorded clip, proceed through an output-free comparison, and reach a running Controller only after the offline result is understood.

Pick a workflow

You do not need to learn every tool. Start with the question you are trying to answer and use the smallest workflow that can answer it.

Question	Start here	Continue only when needed
Did recognition change?	guided-vision-benchmark.py , then benchmark-vision-replay.py	benchmark-palm-anchors.py or one of the isolated tracker experiments
Is movement responsive and stable?	benchmark-native-motion-curve.py	analyze-motion-trace.py or benchmark-direction-response.py
Where is time being spent?	measure-vision-status.py	prepare-end-to-end-session.py , then the bounded latency-tracing workflow
Is camera delivery the bottleneck?	benchmark-camera-pipeline.py	frame-preprocessing or exposure soaks
Does native Super Glove Ball still behave correctly?	build-nestopia-powerglove.sh , then run-nestopia-powerglove-trace.py	the matched direction-response comparison
Are merged controllers assigned, available, and neutral?	The installed virtualglove-controller-route show and check commands	A short physical control test, then bounded status or latency tracing
Is LaunchBox using its intended input route?	Inspect the installed launcher settings and run the loopback RetroPad check	Hotkey inspection or a short physical acceptance run
Could another recognition backend help?	Use the retained Tasks, staggered-tracker, or ncnn comparison	Treat the result as research evidence, not a production switch

Prefer offline tools first. Commands that open a camera or temporarily restart services say so in their help and require deliberate operator steps.

Choose the right download

Download [VirtualGlove-Engineering-Tools.zip](#) from the same GitHub release as the Controller and console installations. Do not mix toolkit and device versions when comparing behaviour.

Also download [VirtualGlove-Engineering-Tools.zip.sha256](#). Verify the download before extracting it:

```
SH
shasum -a 256 -c VirtualGlove-Engineering-Tools.zip.sha256
unzip VirtualGlove-Engineering-Tools.zip
cd VirtualGlove-Engineering-Tools
```

On Linux, [sha256sum](#) -c can be used instead of [shasum](#) -a 256 -c.

First self-check

The initial check needs only Python and does not open a camera, contact a device, install packages, or send controller input:

```
SH
python3 scripts/check-engineering-toolkit.py
```

It verifies the package manifest, paths, Python syntax, direct helper files, and every supported command's side-effect-free help interface. Resolve a failed self-check before collecting evidence.

Create the local environment

Python 3.10 or newer can run offline analysis and video review. The setup script creates [.venv-engineering](#) inside the extracted toolkit and records the exact resolved package versions in [.venv-engineering/virtualglove-engineering-environment.json](#):

```
SH
python3 scripts/setup-engineering-tools.py
```

Activate it on macOS or Linux:

```
SH
. .venv-engineering/bin/activate
```

On Windows PowerShell:

```
POWERSHELL
.\.venv-engineering\Scripts\Activate.ps1
```

MediaPipe comparisons use Python 3.12, matching the supported Controller toolchain:

```
SH
python3.12 scripts/setup-engineering-tools.py --with-mediapipe
```

Rerunning either setup is safe. Verify an existing environment without changing it by adding `--check` and the same MediaPipe option originally used. The check compares the toolkit version, setup mode, Python version, exact resolved package record, and validated direct dependency versions.

The public toolkit does not carry the Controller's ARM64 MediaPipe wheel. A Controller installation already has its validated runtime; do not replace it with a generic workstation package. Use the packaged tools from a workstation for offline analysis and orchestration, or the existing deployed engineering copy on a development Controller when direct camera access is required.

Tool map

The generated `engineering-tools.json` records the exact toolkit version, complete file inventory, and these categories.

Category	Main commands	Changes a running system?
Offline analysis	Latency, motion-trace, direction-response, preprocessing, movement-curve, palm-anchor, post-inference, and diagnostic-overhead tools	No
Camera and recognition	Guided capture, vision replay, camera-pipeline measurement, isolated tracker comparisons, and exposure soaks	May take exclusive ownership of the camera; never sends controller output
Live system tracing	Status measurement, end-to-end preflight, bounded trace management, and native latency sessions	Observation is read-only; trace sessions temporarily restart measured services and restore production when stopped normally
Native and accelerator research	Pinned Nestopia and FCEUmm builders, native-core trace instrumentation, ncnn, and MediaPipe Tasks comparisons	Builds only in a local destination and never installs a core; probes may run caller-supplied binaries or models
Toolkit support	Environment setup and the dependency-free package self-check	Creates or verifies files only inside the toolkit environment

Shared `src/virtualglove` and `config` source is included where tools import or inspect it. Native-emulation research receives the editable VirtualGlove patch and diagnostic header, not the platform release binaries. The package also

contains the project licence, third-party notices, and this guide.

The archive intentionally excludes ROMs, recorded video, trace results, credentials, pairing tokens, player settings, personal calibration, cached models, compiled emulator cores, platform core source archives, and private device data.

Repository-only maintenance tools are excluded: release publishing, device deployment, firmware stamping, package construction, PDF and screenshot generation, enclosure exporting, website generation, platform-native core matrices, and precompiled Matrix release building. The ordinary Controller package already carries the verified firmware; rebuilding it requires the pinned Arduino CLI, Zephyr platform, and libraries from [sketch/sketch.yaml](#) in the full Git repository.

Installed operational tools also remain outside the research archive. In particular, Controller Router configuration and LaunchBox service management belong to the matching console installation. The toolkit explains their read-only diagnostic surfaces, but does not copy a second operational router, receiver, installer, or Windows runtime into a research directory.

Start with offline evidence

Use `--help` on every command before its first run. These examples demonstrate the safest common entry points.

Summarize a Controller motion trace:

```
SH
python scripts/analyze-motion-trace.py TRACE.json --output motion-summary.json
```

Join Controller and receiver latency traces. Add `--core` only when a matching native-core trace exists:

```
SH
python scripts/analyze-latency-trace.py \
  --controller controller.json \
  --receiver receiver.json \
  --output latency-summary.json
```

Prepare a local slow-motion video for reviewed frame annotation:

```
SH
python scripts/analyze-latency-video.py \
  --video recording.mov \
  --protocol smoke \
  --capture-fps 120 \
  --output-dir video-review
```

Replay a retained camera clip through the proven recognition lane:

```
SH
python scripts/benchmark-vision-replay.py \
  clip.avi --quick --output replay.json
```

Recorded clips and generated reports remain local unless the operator moves or uploads them. Review them for faces, rooms, hostnames, or other private material before sharing.

Read-only live observation

The status sampler observes the public Controller status endpoint without opening the camera or changing controller delivery:

```
SH
python scripts/measure-vision-status.py \
  --status-url http://CONTROLLER.local:8088/status?statistics=1 \
  --seconds 60 \
  --phase movement \
  --output status-sample.json
```

The current end-to-end preflight is specifically for a Controller and RetroPie. It checks a development checkout and both devices. It uses SSH but does not change them:

```
SH
python scripts/prepare-end-to-end-session.py \
  --controller-status http://CONTROLLER.local:8088/status?statistics=1 \
  --controller-ssh arduino@CONTROLLER.local \
  --retropie-ssh pi@RETROPIE.local \
  --output-dir preflight
```

Use dedicated SSH identities through the corresponding identity options. Do not place private keys inside the toolkit directory or an evidence bundle.

Do not substitute Recalbox, Batocera, or LaunchBox details into the `--retropie-ssh` option. Their service layouts, persistence rules, and input routes differ. Use the installed platform checks below, and record that the general RetroPie preflight was not run.

Diagnose Controller Router

Controller Router is an installed console subsystem, not an offline research tool. Its read-only commands are nevertheless the best starting point when a physical controller is missing, assigned to the wrong player, or translated differently after an EmulationStation remap.

Do not run these commands while changing assignments in Setup. Finish or cancel that operation first. `show` reports the saved revision, assignments, and inventory. `check` adds current availability and output validation without changing the configuration.

On RetroPie:

```
SH
sudo /opt/virtualglove/bin/virtualglove-controller-router show
sudo /opt/virtualglove/bin/virtualglove-controller-router check
```

On Recalbox:

```
SH
sh /recalbox/share/system/virtualglove/scripts/virtualglove-controller-router show
sh /recalbox/share/system/virtualglove/scripts/virtualglove-controller-router check
```

On Batocera:

```
SH
sh /userdata/system/virtualglove/scripts/virtualglove-controller-router show
sh /userdata/system/virtualglove/scripts/virtualglove-controller-router check
```

The Recalbox and Batocera launchers are invoked with [sh](#) because their persistent shares do not permit direct program execution. Run the commands from the console itself. Do not copy the private router document into a public evidence bundle: friendly controller names and stable identity suffixes can identify a particular cabinet.

For a physical acceptance check, use **Test controls** in Setup or the local [virtualglove-controller-router setup](#) screen. The test is bounded and does not save unless the operator explicitly chooses **Save and verify**. Capture:

- platform and installed VirtualGlove version;
- router revision and enabled player slots;
- each assigned source's friendly name, stable identity suffix, mapping status, and connection state;
- the VirtualGlove player assignment;
- whether the original controllers still navigate EmulationStation;
- the emulator and core used for the game; and
- neutral release after the game exits or a source disconnects.

An unavailable saved source is evidence, not permission to substitute another device. Controller Router deliberately refuses to guess when identical devices or incomplete mappings are ambiguous.

Diagnose LaunchBox input

LaunchBox does not use Controller Router. VirtualGlove reaches ordinary RetroArch games through a managed loopback Network RetroPad while the physical XInput controller remains independent. Native Super Glove Ball uses its separate Nestopia (VirtualGlove) path.

With LaunchBox, Big Box, and RetroArch closed, inspect the installed route in PowerShell:

```
POWERSHELL
$settings = Get-Content "$env:LOCALAPPDATA\VirtualGlove\data\launcher.json" |
    ConvertFrom-Json
$settings | Select-Object uno_q, input_route, retroarch_remote_port
```

[input_route](#) should be [network-retropad](#), and the port should be in the dynamic range from 49152 through 65535. Then prove that the managed Python runtime can deliver a correctly sized neutral packet over loopback:

```
POWERSHELL
& "$env:LOCALAPPDATA\VirtualGlove\runtime\Scripts\python.exe" `
    -m virtualglove.retroarch_remote `
    --check-loopback $settings.retroarch_remote_port
```

The check requires RetroArch to be closed so it can bind the selected port. It does not press a game control. The installer separately verifies that Windows Firewall blocks LocalSubnet access to this port; do not weaken that rule to make a diagnostic pass. Record the route, port, firewall result, FCEUmm or native-core selection, and whether VirtualGlove, the physical controller, and the real keyboard each work after repeated launches.

Guided camera capture

Stop normal vision before allowing an engineering tool to own the camera. Keep controller output stopped and ensure no game depends on the camera during the test.

The single supported recorder provides a local live preview and user-paced cues. It replaced the older fixed-duration recorder so the operator can verify each pose instead of racing a timer:

```
SH
python scripts/guided-vision-benchmark.py \
  --camera auto \
  --protocol tracking \
  --output guided-tracking.avi
```

The page address is printed when the recorder starts. Finish or interrupt the recorder normally so it releases the camera. Restart normal vision and confirm the Dashboard preview before playing.

Camera pipeline and exposure tools require explicit acknowledgement flags. Those flags confirm that the operator has stopped the ordinary camera worker; they are not shortcuts around that safety step.

Tracing a running two-device system

[manage-latency-traces.py](#) temporarily recreates measured services with bounded diagnostic tracing. Use its [start](#) and [stop](#) subcommands as one matched pair. The stop action restores production before collecting finalized traces.

Before starting:

1. Confirm Controller and target Linux console SSH access.
2. Confirm the expected game and emulator are selected.
3. Stop if either device is already unhealthy.
4. Choose a short bounded duration.
5. Keep a separate terminal ready to run the stop command.

Do not publish raw trace bundles. Although controller packets contain no camera frames, reports can contain host information, paths, timings, and configuration details.

Native emulator research

Build the pinned, patched Nestopia core in a local directory without installing it:

```
SH
sh scripts/build-nestopia-powerglove.sh build/nestopia-research
```

Build a stock FCEUmm comparison core when a matched joystick test needs it:

```
SH
sh scripts/build-fceumm-benchmark.sh build/fceumm-research
```


Both builders fetch pinned upstream source. Review their output and licences; do not substitute their local products for platform-packaged release cores.

The full repository also contains target-specific builders and verifiers for RetroPie, Recalbox, Batocera, and LaunchBox. Those commands produce release artefacts against platform toolchains, manifests, and ABI checks, so they are maintainer tools rather than Engineering Toolkit experiments. Use a complete Git checkout and the Configuration Reference when updating a platform matrix.

`run-nestopia-powerglove-trace.py` loads a caller-supplied libretro core and ROM without installing either one. The toolkit never supplies commercial ROMs. Keep ROM and scratch paths outside the extracted toolkit, and remove scratch copies when the run finishes.

The ncnn and MediaPipe Tasks tools are retained for reproducible comparison; they are not production alternatives. A faster isolated layer or delegate is not sufficient evidence. Any proposed runtime must preserve recognition, continuity, ordering, thermal stability, and complete camera-to-game latency.

The toolkit no longer includes the old fixed recorder, `min/boost` filename matrix, or earlier ideal-step sampler. Their assumptions belonged to retired motion experiments. Guided capture, labelled replay evidence, and the bounded native-motion curve are the maintained replacements.

Evidence checklist

Every retained result should record:

- VirtualGlove release and toolkit version
- Controller and console build identifiers
- Console platform, emulator/core, active input route, and Controller Router revision or LaunchBox RetroPad port when applicable
- Camera model, reader, frame rate, buffer count, exposure, and gain
- Python and resolved package environment record
- Exact command and input-file digests
- Whether Dashboard preview and statistics were open
- Test duration, operator cues, limitations, and unexpected interruptions
- Aggregate output filename and SHA-256 digest

Never treat results from different clips or physical movements as a controlled A/B comparison. Never combine latency percentiles measured on clocks that were not synchronized.

Cleanup and recovery

Generated environments, clips, traces, and reports are not managed by the VirtualGlove installer.

- Delete and recreate `.venv-engineering` when changing Python families.
- Keep one known-good environment record with important evidence.
- Move valuable clips and aggregate reports into a clearly named private evidence directory.
- Remove abandoned scratch directories and duplicate extracted toolkits.
- Do not clear the Controller's active `uv` cache merely to make space; it supports reliable offline worker restarts.
- Archive old development-device evidence before removing it.
- After any camera-owning experiment, confirm the Dashboard preview and normal game control before declaring recovery complete.

For active file locations and option definitions, see the online [Configuration Reference](#). For the current runtime and device boundaries, see [Architecture and flows](#). For why specific experiments were retained or rejected, see the [Engineering Journey](#).